

Documentation: Portfolio Part-1

Alexander Ricciardi

Colorado State University Global

CSC450: Programming III

Professor: Reginald Haseltine

November 24, 2024

Documentation: Portfolio Part-1

This documentation is part of the Module-7 Portfolio Milestone from CSC450: Programming III at Colorado State University Global. The program's name is Thread Counting Synchronization. It provides an overview of the program's functionality and testing scenarios, including console output screenshots. The program is coded in C++ 17.

The Assignment Direction:

Portfolio Project: Part 1

For your Portfolio Project, you will demonstrate an understanding of the various concepts discussed in each module. For the first part of your Portfolio Project, you will create a C++ application that will exhibit concurrency concepts. Your application should create two threads that will act as counters. One thread should count up to 20. Once thread one reaches 20, then a second thread should be used to count down to 0. For your created code, provide a detailed analysis of appropriate concepts that could impact your application. Specifically, address:

- Performance issues with concurrency
- Vulnerabilities exhibited with use of strings
- Security of the data types exhibited.

Compile and submit your pseudocode, source code, and screenshots of the application executing the application, the results and your GIT repository in a single document.

To receive full credit for the packaging requirements for your Critical Thinking and Portfolio assignments you must:

- 1) Put your C++ source code in .cpp text files. Note that I execute all your programs to check them out.
- 2) In a Word or PDF "documentation" file, labeled as such, put a copy of your C++ source code and execution output screen snapshots.
- 3) Some positive evidence that you've definitely stored your source code in a GitHub repository on GitHub.com.
- 4) Include a detailed analysis paper in APA Edition 7 format of the important concepts of concurrency with C++ to cover in detail performance issues, string vulnerabilities, and security of data types. Here's a link to the school's Writing Center where you can find the relevant APA Edition 7 requirements you need to follow -> <https://csuglobal.libguides.com/writingcenterLinks> to an external site.
- 5) Put all your files into a single .zip file, and submit ONLY that .zip file for grading. Do not submit any additional separate files.

My notes:

- The simple C++ console application is in file **PF1-Thread Counting Synchronization.cpp**
- The program follows the following SEI CERT C/C++ Coding Standard:
 - CON50-CPP. Do not destroy a mutex while it is locked
 - CON51-CPP. Ensure actively held locks are released on exceptional conditions
 - CON52-CPP. Prevent data races when accessing bit-fields from multiple threads
 - CON54-CPP. Wrap functions that can spuriously wake up in a loop
 - CON55-CPP. Preserve thread safety and liveness when using condition variables
 - ERR50-CPP. Do not abruptly terminate the program
 - ERR51-CPP. Handle all exceptions
 - ERR55-CPP. Honor Exception Specifications
 - STR50-CPP. Guarantee that storage for strings has sufficient space
 - STR51-CPP. Do not attempt to create a std::string from a null pointer

- STR52-CPP. Use valid references, pointers, and iterators to reference elements of a `basic_string`

Program Description:

This program demonstrates the use of threads and how to synchronize them using mutexes and condition variables.

Thread 1 counts up from 0 to a maximum count, while Thread 2 waits until Thread 1 completes, and then counts down from the maximum count to 0.

Git Repository

I use [GitHub](#) as my Distributed Version Control System (DVCS), the following is a link to my GitHub, [Omegapy](#).

My GitHub repository that is used to store this assignment is named [My-Academics-Portfolio](#).

The link to this specific assignment is:

<https://github.com/Omegapy/My-Academics-Portfolio/tree/main/Programming-3-CSC450/Portfolio-Part-1>

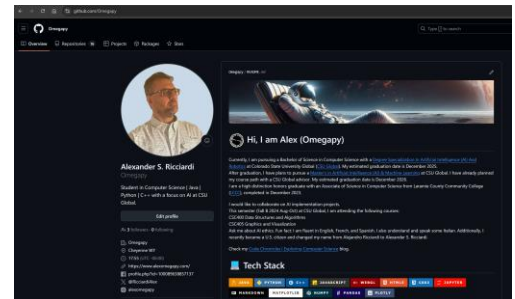
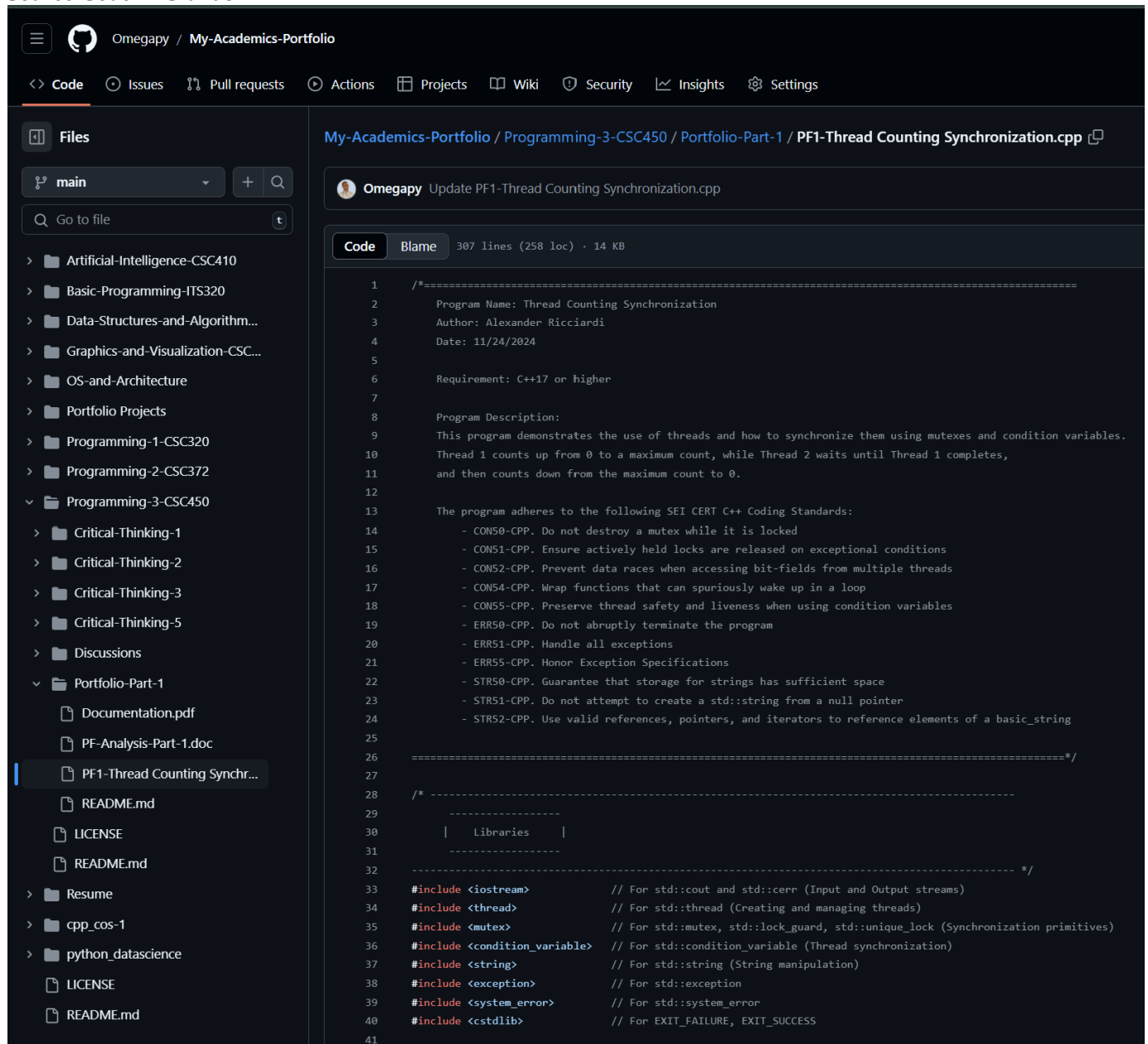


Image of the source code in the GitHub: see next page

Figure 1
Source Code in GitHub



Continue next page

Go to file

- Artificial-Intelligence-CSC410
- Basic-Programming-ITS320
- Data-Structures-and-Algorithm...
- Graphics-and-Visualization-CSC...
- OS-and-Architecture
- Portfolio Projects
- Programming-1-CSC320
- Programming-2-CSC372
- Programming-3-CSC450
 - Critical-Thinking-1
 - Critical-Thinking-2
 - Critical-Thinking-3
 - Critical-Thinking-5
 - Discussions
 - Portfolio-Part-1
 - PF1-Thread Counting Synchron...
 - README.md
 - LICENSE
 - README.md
 - Resume
 - cpp_cos-1
 - python_datascience
 - LICENSE
 - README.md

```

42  /* -----
43  | Global Variables |
44  ----- */
45
46  // Banner - multi-line string
47  const std::string banner = R"(
48  *****
49  * Thread Counting Synchronization *
50  *****
51  )";
52
53  // Mutex to protect shared data
54  std::mutex mtx;
55
56  // Condition variable for thread signaling
57  std::condition_variable cv;
58
59  // Flag to indicate completion of counting up
60  bool isCountingUpDone = false;
61
62  /* -----
63  | Function Declarations |
64  ----- */
65
66  /**
67   * Counts up from 0 to maxCount.
68   *
69   * Handles Rules:
70   * - CON50-CPP: Do not destroy a mutex while it is locked
71   * - CON51-CPP: Ensure actively held locks are released on exceptional conditions
72   * - CON52-CPP: Prevent data races when accessing bit-fields from multiple threads
73   * - CON54-CPP: Wrap functions that can spuriously wake up in a loop
74   * - STR51-CPP: Do Not Attempt to Create a std::string from a Null Pointer
75   * - STR52-CPP: Use Valid References, Pointers, and Iterators to Reference Elements of a basic_string
76   * - ERR51-CPP: Handle All Exceptions
77   * - ERR55-CPP: Honor Exception Specifications
78   *
79   * @param threadName thread's name.
80   * @param counter the data to be modify by the thread.
81   */
82  void countUp(const std::string& threadName, int& counter);
83
84  /**
85   * Counts down from startCount to 0.
86   *
87   * Handles Rules:
88   * - CON54-CPP: Wrap Functions That Can Spuriously Wake Up in a loop
89   * - CON55-CPP: Preserve Thread Safety and Liveness When Using Condition Variables
90   * - ERR51-CPP: Handle All Exceptions
91   * - ERR55-CPP: Honor Exception Specifications
92   * - STR52-CPP: Use Valid References, Pointers, and Iterators to Reference Elements of a basic_string
93   * - STR51-CPP: Do Not Attempt to Create a std::string from a Null Pointer
94   *
95   * @param threadName thread's name.
96   * @param counter The data to be modify by the thread.
97   */
98  void countDown(const std::string& threadName, int& counter);
99

```

Continue next page

Q Go to file

t

> Artificial-Intelligence-CSC410

> Basic-Programming-ITS320

> Data-Structures-and-Algorithm...

> Graphics-and-Visualization-CSC...

> OS-and-Architecture

> Portfolio Projects

> Programming-1-CSC320

> Programming-2-CSC372

> Programming-3-CSC450

> Critical-Thinking-1

> Critical-Thinking-2

> Critical-Thinking-3

> Critical-Thinking-5

> Discussions

> Portfolio-Part-1

PF1-Thread Counting Synchron...

README.md

LICENSE

README.md

> Resume

> cpp_cos-1

> python_datascience

LICENSE

README.md

104 // =====

105 /* -----

106 -----

107 | Main Function |

108 -----

109

110 The main function creates threads for counting up and down.

111

112 Handles Rules:

113 - CON50-CPP: Do Not Destroy a Mutex While It Is Locked

114 - ERR51-CPP: Handle All Exceptions

115 - ERR50-CPP: Do Not Abruptly Terminate the Program

116 - ERR55-CPP: Honor Exception Specifications

117 - STR50-CPP: Guarantee that storage for strings has sufficient space

118

119 ----- */

120 // =====

121 int main() {

122 // Define thread names for clarity in output

123 // (STR50-CPP: Guarantee that storage for strings has sufficient space)

124 std::string thread1Name = "Thread 1";

125 std::string thread2Name = "Thread 2";

126

127 // Maximum count for Thread 1

128 int counter = -1;

129

130 try {

131 // Output banner

132 std::cout << banner << std::endl;

133

134 // Create Thread 1 (Counting Up)

135 std::thread thread1(countUp, thread1Name, std::ref(counter));

136

137 // Create Thread 2 (Counting Down)

138 std::thread thread2(countDown, thread2Name, std::ref(counter));

139

140 // Join threads to ensure they have completed execution

141 // (CON50-CPP: Do not destroy a mutex while it is locked)

142 thread1.join();

143 thread2.join();

144 }

145 catch (const std::system_error& e) {

146 // Ensure actively held locks are released on exceptional conditions (ERR51-CPP)

147 std::cerr << "Thread system error: " << e.what() << std::endl;

148 return EXIT_FAILURE; // Do not abruptly terminate the program (ERR50-CPP)

149 }

150 catch (const std::exception& e) {

151 // Handle any other standard exceptions

152 std::cerr << "Exception: " << e.what() << std::endl;

153 return EXIT_FAILURE; // Do not abruptly terminate the program (ERR50-CPP)

154 }

155 catch (...) {

156 // Catch-all handler for any other exceptions

157 std::cerr << "Unknown exception occurred." << std::endl;

158 return EXIT_FAILURE; // Do not abruptly terminate the program (ERR50-CPP)

159 }

160

161 // Final output indicating completion

162 // (STR52-CPP: Use valid references, pointers, and iterators to reference elements of a basic_string)

163 std::cout << "\nBoth threads have completed their counting without errors." << std::endl;

164

165 return EXIT_SUCCESS;

166 }

Continue next page

Go to file
t

- Artificial-Intelligence-CSC410
- Basic-Programming-ITS320
- Data-Structures-and-Algorith...
- Graphics-and-Visualization-CS...
- OS-and-Architecture
- Portfolio Projects
- Programming-1-CSC320
- Programming-2-CSC372
- Programming-3-CSC450
- Critical-Thinking-1
- Critical-Thinking-2
- Critical-Thinking-3
- Critical-Thinking-5
- Discussions
- Portfolio-Part-1
 - PF1-Thread Counting Synchron...
 - README.md
 - LICENSE
 - README.md
 - Resume
 - cpp_cos-1
 - python_datascience
 - LICENSE
 - README.md

```

167
168 // =====
169 /* =====
170 | Function Definitions |
171 | =====
172 | =====
173 | ===== */
174 // =====
175
176 /**
177  * Counts up from 0 to maxCount.
178  *
179  * Handles Rules:
180  * - CON50-CPP: Do not destroy a mutex while it is locked
181  * - CON51-CPP: Ensure actively held locks are released on exceptional conditions
182  * - CON54-CPP: Wrap functions that can spuriously wake up in a loop
183  * - STR51-CPP: Do Not Attempt to Create a std::string from a Null Pointer
184  * - STR52-CPP: Use Valid References, Pointers, and Iterators to Reference Elements of a basic_string
185  * - ERR51-CPP: Handle All Exceptions
186  * - ERR55-CPP: Honor Exception Specifications
187  *
188  * @param threadName thread's name.
189  * @param counter the data to be modify by the thread.
190  */
191 void countUp(const std::string& threadName, int& counter) {
192     try {
193         // Ensure that threadName is a valid, non-null string (STR51-CPP)
194
195         std::cout << "\n--- Thread 1 is live ---" << std::endl;
196
197
198         for (int i = 0; i <= 20; i++) {
199             // Simulate some work with a sleep
200             // Note: The mutex is not locked, other threads can access the shared data (counter) during this time
201             std::this_thread::sleep_for(std::chrono::milliseconds(100));
202
203             // Using std::lock_guard ensures that the mutex is automatically released when the scope ends,
204             // even if an exception is thrown (CON51-CPP)
205             // std::lock_guard is used to to lock exactly one mutex for an entire scope
206             // (Using RAII to manage mutex unlocking automatically - Related to Unlocking a mutex via RAII)
207             { // Scope for lock_guard allows for automatic unlocking of the mutex when the scope ends
208                 std::lock_guard<std::mutex> lock(mtx); // (CON51-CPP: Automatically unlocks mutex) (at the end of the scope)
209
210                 if (i == 0) {
211                     std::cout << "\n--- Counting Up Thread 1 ---" << std::endl;
212                 }
213
214                 // Use valid references to elements of a basic_string (STR52-CPP)
215                 std::cout << threadName << " counting up: " << ++counter << std::endl;
216             } // RAII ensures mutex is unlocked automatically when the scope ends
217         }
218
219         // After counting up is done, set the flag
220         { // Lock mutex before modifying shared flag to prevent data races (CON52-CPP)
221             std::lock_guard<std::mutex> lock(mtx); // (CON51-CPP)
222             isCountingUpDone = true;
223         }
224
225         // Notify one waiting thread to start counting down
226         // (CON54-CPP: Wrap functions that can spuriously wake up in a loop)
227         cv.notify_one();
228
229         // (CON50-CPP: Do not destroy a mutex while it is locked)
230         // The mutex mtx remains valid, it is declared globally and threads are properly joined before the program is terminated.
231     }
232     catch (const std::exception& e) {
233         // Handle exceptions within the thread (ERR51-CPP)
234         std::cerr << threadName << " encountered an exception: " << e.what() << std::endl;
235     }
236     catch (...) {
237         std::cerr << threadName << " encountered an unknown exception." << std::endl;
238     }
239 }
240
241 // =====

```

Continue next page

Critical-Thinking-2

Critical-Thinking-3

Critical-Thinking-5

Discussions

Portfolio-Part-1

Documentation.pdf

PF1-Analysis-Part-1.doc

PF1-Thread Counting Synch...

README.md

LICENSE

README.md

Resume

cpp_cos-1

python_datascience

LICENSE

README.md

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

/**

* Counts down from startCount to 0.

*

* Handles Rules:

* - CON54-CPP: Wrap Functions That Can Spuriously Wake Up in a Loop

* - CON55-CPP: Preserve Thread Safety and Liveness When Using Condition Variables

* - ERR51-CPP: Handle All Exceptions

* - ERR55-CPP: Honor Exception Specifications

* - STR52-CPP: Use Valid References, Pointers, and Iterators to Reference Elements of a basic_string

* - STR51-CPP: Do Not Attempt to Create a std::string from a Null Pointer

*

* @param threadName thread's name.

* @param counter The data to be modify by the thread.

*/

void countdown(const std::string& threadName, int& counter) {

try {

// Ensure that threadName is a valid, non-null string (STR51-CPP)

std::cout << "\n-- Thread 2 is live ---" << std::endl;

// std::unique_lock<std::mutex> is used to lock the mutex and work with the condition variable (wait).

// Provides more flexibility compared to std::lock_guard, such as manual unlocking.

std::unique_lock<std::mutex> lock(mtx); // (CON51-CPP: Ensures mutex is unlocked on exceptions)

// Wait until isCountingUpDone is true

// (CON54-CPP: Wrap functions that can spuriously wake up in a loop)

cv.wait(lock, [] { return isCountingUpDone; }); // Mutex is unlocked during wait

std::cout << "\n--- Counting down Thread 2 ---" << std::endl;

// Unlock the mutex before 'Simulate' some work with a sleep in the for-loop

// the mutex was locked by the unique_lock above to create a condition variable wait

// (Explicit Unlock - Using std::unique_lock)

lock.unlock();

// Start counting down

for (int i = counter; i >= 0; --i) {

// Simulate some work with a sleep

// Note: The mutex is not locked, other threads can access the shared data (counter) during this time

std::this_thread::sleep_for(std::chrono::milliseconds(100));

{ // Scope for lock_guard allows for automatic unlocking of the mutex when the scope ends

std::lock_guard<std::mutex> guard(mtx); // (CON51-CPP: Automatically unlocks mutex)

// Use valid references to elements of a basic_string (STR52-CPP)

std::cout << threadName << " counting down: " << counter-- << std::endl;

} // RAII ensures mutex is unlocked automatically when the scope ends

}

// Ensure that threads terminate properly, maintaining liveness and thread safety

// (CON55-CPP: Preserve thread safety and liveness when using condition variables)

}

catch (const std::exception& e) {

// Handle exceptions within the thread (ERR51-CPP)

std::cerr << threadName << " encountered an exception: " << e.what() << std::endl;

}

catch (...) {

std::cerr << threadName << " encountered an unknown exception." << std::endl;

}

}

// -----

// End of Program

Next page Source Code in IDE

Figure 2
Source Code in IDE

```

PF1-Thread C...onization.cpp  X
Module-7 Portfolio Milestone (Global Scope)

1  /*=====
2  Program Name: Thread Counting Synchronization
3  Author: Alexander Ricciardi
4  Date: 11/24/2024
5
6  Requirement: C++17 or higher
7
8  Program Description:
9  This program demonstrates the use of threads and how to synchronize them using mutexes and condition variables.
10 Thread 1 counts up from 0 to a maximum count, while Thread 2 waits until Thread 1 completes,
11 and then counts down from the maximum count to 0.
12
13 The program adheres to the following SEI CERT C++ Coding Standards:
14 - CON50-CPP. Do not destroy a mutex while it is locked
15 - CON51-CPP. Ensure actively held locks are released on exceptional conditions
16 - CON52-CPP. Prevent data races when accessing bit-fields from multiple threads
17 - CON54-CPP. Wrap functions that can spuriously wake up in a loop
18 - CON55-CPP. Preserve thread safety and liveness when using condition variables
19 - ERR50-CPP. Do not abruptly terminate the program
20 - ERR51-CPP. Handle all exceptions
21 - ERR55-CPP. Honor Exception Specifications
22 - STR50-CPP. Guarantee that storage for strings has sufficient space
23 - STR51-CPP. Do not attempt to create a std::string from a null pointer
24 - STR52-CPP. Use valid references, pointers, and iterators to reference elements of a basic_string
25
26 =====*/
27
28 /*-----
29 | Libraries |
30 |-----*/
31
32 #include <iostream> // For std::cout and std::cerr (Input and Output streams)
33 #include <thread> // For std::thread (Creating and managing threads)
34 #include <mutex> // For std::mutex, std::lock_guard, std::unique_lock (Synchronization primitives)
35 #include <condition_variable> // For std::condition_variable (Thread synchronization)
36 #include <string> // For std::string (String manipulation)
37 #include <exception> // For std::exception
38 #include <system_error> // For std::system_error
39 #include <cstdlib> // For EXIT_FAILURE, EXIT_SUCCESS
40
41 /*-----
42 | Global Variables |
43 |-----*/
44
45
46
47
48 // Banner - multi-line string
49 const std::string banner = R"(
50 *****
51 * Thread Counting Synchronization *
52 *****
53 )";
54
55 // Mutex to protect shared data
56 std::mutex mtx;
57
58 // Condition variable for thread signaling
59 std::condition_variable cv;
60
61 // Flag to indicate completion of counting up
62 bool isCountingUpDone = false;
63

```

Continue next page

```

64  |  /* -----
65  |  |
66  |  |   Function Declarations   |
67  |  |
68  |  | ----- */
69  |
70  |  /**
71  |  | * Counts up from 0 to maxCount.
72  |  | *
73  |  | * Handles Rules:
74  |  | *   - CON50-CPP: Do not destroy a mutex while it is locked
75  |  | *   - CON51-CPP: Ensure actively held locks are released on exceptional conditions
76  |  | *   - CON52-CPP: Prevent data races when accessing bit-fields from multiple threads
77  |  | *   - CON54-CPP: Wrap functions that can spuriously wake up in a loop
78  |  | *   - STR51-CPP: Do Not Attempt to Create a std::string from a Null Pointer
79  |  | *   - STR52-CPP: Use Valid References, Pointers, and Iterators to Reference Elements of a basic_string
80  |  | *   - ERR51-CPP: Handle All Exceptions
81  |  | *   - ERR55-CPP: Honor Exception Specifications
82  |  | *
83  |  | * @param threadName thread's name.
84  |  | * @param counter the data to be modify by the thread.
85  |  | */
86  |  void countUp(const std::string& threadName, int& counter);
87  |
88  |  /**
89  |  | * Counts down from startCount to 0.
90  |  | *
91  |  | * Handles Rules:
92  |  | *   - CON54-CPP: Wrap Functions That Can Spuriously Wake Up in a Loop
93  |  | *   - CON55-CPP: Preserve Thread Safety and Liveness When Using Condition Variables
94  |  | *   - ERR51-CPP: Handle All Exceptions
95  |  | *   - ERR55-CPP: Honor Exception Specifications
96  |  | *   - STR52-CPP: Use Valid References, Pointers, and Iterators to Reference Elements of a basic_string
97  |  | *   - STR51-CPP: Do Not Attempt to Create a std::string from a Null Pointer
98  |  | *
99  |  | * @param threadName thread's name.
100 |  | * @param counter The data to be modify by the thread.
101 |  | */
102 |  void countDown(const std::string& threadName, int& counter);
103 |

```

Continue next page

```

104 // =====
105 /*
106 |-----|
107 | Main Function |
108 |-----|
109
110 The main function creates threads for counting up and down.
111
112 Handles Rules:
113 - CON50-CPP: Do Not Destroy a Mutex While It Is Locked
114 - ERR51-CPP: Handle All Exceptions
115 - ERR50-CPP: Do Not Abruptly Terminate the Program
116 - ERR55-CPP: Honor Exception Specifications
117 - STR50-CPP: Guarantee that storage for strings has sufficient space
118
119 ----- */
120 // =====
121 int main() {
122     // Define thread names for clarity in output
123     // (STR50-CPP: Guarantee that storage for strings has sufficient space)
124     std::string thread1Name = "Thread 1";
125     std::string thread2Name = "Thread 2";
126
127     // Maximum count for Thread 1
128     int counter = -1;
129
130     try {
131         // Output banner
132         std::cout << banner << std::endl;
133
134         // Create Thread 1 (Counting Up)
135         std::thread thread1(countUp, thread1Name, std::ref(counter));
136
137         // Create Thread 2 (Counting Down)
138         std::thread thread2(countDown, thread2Name, std::ref(counter));
139
140         // Join threads to ensure they have completed execution
141         // (CON50-CPP: Do not destroy a mutex while it is locked)
142         thread1.join();
143         thread2.join();
144     }
145     catch (const std::system_error& e) {
146         // Ensure actively held locks are released on exceptional conditions (ERR51-CPP)
147         std::cerr << "Thread system error: " << e.what() << std::endl;
148         return EXIT_FAILURE; // Do not abruptly terminate the program (ERR50-CPP)
149     }
150     catch (const std::exception& e) {
151         // Handle any other standard exceptions
152         std::cerr << "Exception: " << e.what() << std::endl;
153         return EXIT_FAILURE; // Do not abruptly terminate the program (ERR50-CPP)
154     }
155     catch (...) {
156         // Catch-all handler for any other exceptions
157         std::cerr << "Unknown exception occurred." << std::endl;
158         return EXIT_FAILURE; // Do not abruptly terminate the program (ERR50-CPP)
159     }
160
161     // Final output indicating completion
162     // (STR52-CPP: Use valid references, pointers, and iterators to reference elements of a basic_string)
163     std::cout << "\nBoth threads have completed their counting without errors." << std::endl;
164
165     return EXIT_SUCCESS;
166 }

```

Continue next page

```

167 // =====
168 // */
169 |-----|
170 |   Function Definitions   |
171 |-----|
172 |
173 |-----|
174 // =====
175
176 /**
177  * Counts up from 0 to maxCount.
178  *
179  * Handles Rules:
180  * - CONS0-CPP: Do not destroy a mutex while it is locked
181  * - CONS1-CPP: Ensure actively held locks are released on exceptional conditions
182  * - CONS4-CPP: Wrap functions that can spuriously wake up in a loop
183  * - STR51-CPP: Do Not Attempt to Create a std::string from a Null Pointer
184  * - STR52-CPP: Use Valid References, Pointers, and Iterators to Reference Elements of a basic_string
185  * - ERR51-CPP: Handle All Exceptions
186  * - ERR55-CPP: Honor Exception Specifications
187  *
188  * @param threadName thread's name.
189  * @param counter the data to be modify by the thread.
190  */
191 void countUp(const std::string& threadName, int& counter) {
192     try {
193         // Ensure that threadName is a valid, non-null string (STR51-CPP)
194
195         std::cout << "\n--- Thread 1 is live ---" << std::endl;
196
197         for (int i = 0; i <= 20; i++) {
198             // Simulate some work with a sleep
199             // Note: The mutex is not locked, other threads can access the shared data (counter) during this time
200             std::this_thread::sleep_for(std::chrono::milliseconds(100));
201
202             // Using std::lock_guard ensures that the mutex is automatically released when the scope ends,
203             // even if an exception is thrown (CONS1-CPP)
204             // std::lock_guard is used to to lock exactly one mutex for an entire scope
205             // (Using RAII to manage mutex unlocking automatically - Related to Unlocking a mutex via RAII)
206             { // Scope for lock_guard allows for automatic unlocking of the mutex when the scope ends
207                 std::lock_guard<std::mutex> lock(mtx); // (CONS1-CPP: Automatically unlocks mutex) (at the end of the scope)
208
209                 if (i == 0) {
210                     std::cout << "\n--- Counting Up Thread 1 ---" << std::endl;
211                 }
212
213                 // Use valid references to elements of a basic_string (STR52-CPP)
214                 std::cout << threadName << " counting up: " << ++counter << std::endl;
215             } // RAII ensures mutex is unlocked automatically when the scope ends
216
217             // After counting up is done, set the flag
218             { // Lock mutex before modifying shared flag to prevent data races (CONS2-CPP)
219                 std::lock_guard<std::mutex> lock(mtx); // (CONS1-CPP)
220                 isCountingUpDone = true;
221             }
222
223             // Notify one waiting thread to start counting down
224             // (CONS4-CPP: Wrap functions that can spuriously wake up in a loop)
225             cv.notify_one();
226
227             // (CONS0-CPP: Do not destroy a mutex while it is locked)
228             // The mutex mtx remains valid, it is declared globally and threads are properly joined before the program is terminated.
229         }
230     } catch (const std::exception& e) {
231         // Handle exceptions within the thread (ERR51-CPP)
232         std::cerr << threadName << " encountered an exception: " << e.what() << std::endl;
233     } catch (...) {
234         std::cerr << threadName << " encountered an unknown exception." << std::endl;
235     }
236 }
237
238
239

```

Continue next page

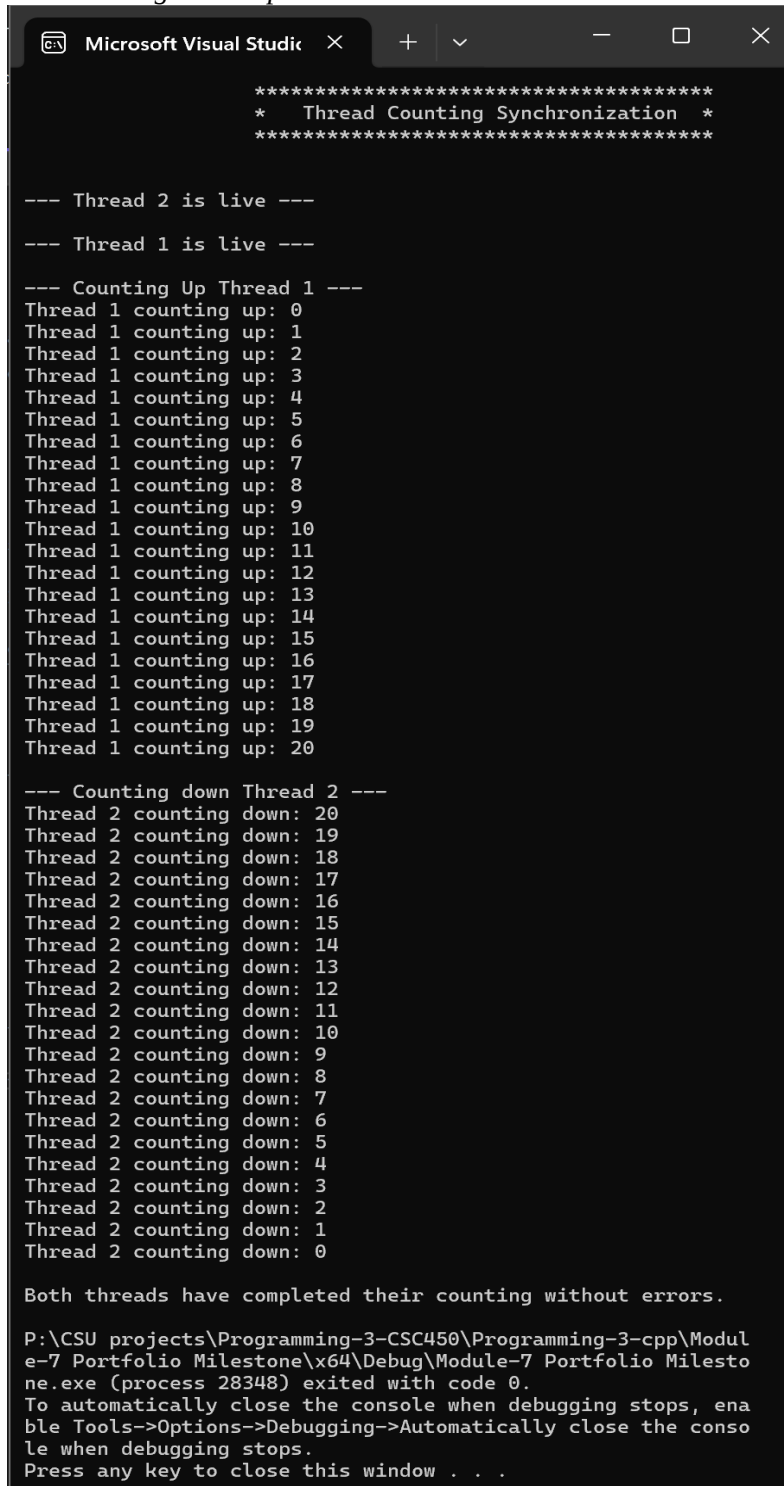
```

240
241 // -----
242
243 /**
244  * Counts down from startCount to 0.
245  *
246  * Handles Rules:
247  *   - CON54-CPP: Wrap Functions That Can Spuriously Wake Up in a Loop
248  *   - CON55-CPP: Preserve Thread Safety and Liveness When Using Condition Variables
249  *   - ERR51-CPP: Handle All Exceptions
250  *   - ERR55-CPP: Honor Exception Specifications
251  *   - STR52-CPP: Use Valid References, Pointers, and Iterators to Reference Elements of a basic_string
252  *   - STR51-CPP: Do Not Attempt to Create a std::string from a Null Pointer
253  *
254  * @param threadName thread's name.
255  * @param counter The data to be modify by the thread.
256  */
257 void countdown(const std::string& threadName, int& counter) {
258     try {
259         // Ensure that threadName is a valid, non-null string (STR51-CPP)
260
261         std::cout << "\n--- Thread 2 is live ---" << std::endl;
262
263         // std::unique_lock<std::mutex> is used to lock the mutex and work with the condition variable (wait).
264         // Provides more flexibility compared to std::lock_guard, such as manual unlocking.
265         std::unique_lock<std::mutex> lock(mtx); // (CON51-CPP: Ensures mutex is unlocked on exceptions)
266
267         // Wait until isCountingUpDone is true
268         // (CON54-CPP: Wrap functions that can spuriously wake up in a loop)
269         cv.wait(lock, [] { return isCountingUpDone; }); // Mutex is unlocked during wait
270
271         std::cout << "\n--- Counting down Thread 2 ---" << std::endl;
272
273         // Unlock the mutex before 'Simulate' some work with a sleep in the for-loop
274         // the mutex was locked by the unique_lock above to create a condition variable wait
275         // (Explicit Unlock - Using std::unique_lock)
276         lock.unlock();
277
278         // Start counting down
279         for (int i = counter; i >= 0; --i) {
280
281             // Simulate some work with a sleep
282             // Note: The mutex is not locked, other threads can access the shared data (counter) during this time
283             std::this_thread::sleep_for(std::chrono::milliseconds(100));
284
285             { // Scope for lock_guard allows for automatic unlocking of the mutex when the scope ends
286                 std::lock_guard<std::mutex> guard(mtx); // (CON51-CPP: Automatically unlocks mutex)
287
288                 // Use valid references to elements of a basic_string (STR52-CPP)
289                 std::cout << threadName << " counting down: " << counter-- << std::endl;
290             } // RAII ensures mutex is unlocked automatically when the scope ends
291
292         }
293
294         // Ensure that threads terminate properly, maintaining liveness and thread safety
295         // (CON55-CPP: Preserve thread safety and liveness when using condition variables)
296     }
297     catch (const std::exception& e) {
298         // Handle exceptions within the thread (ERR51-CPP)
299         std::cerr << threadName << " encountered an exception: " << e.what() << std::endl;
300     }
301     catch (...) {
302         std::cerr << threadName << " encountered an unknown exception." << std::endl;
303     }
304 }
305
306 // -----
307 // End of Program

```

Next page Console output

Figure 3
Console Program Output



```

Microsoft Visual Studio
*****
*   Thread Counting Synchronization   *
*****

--- Thread 2 is live ---

--- Thread 1 is live ---

--- Counting Up Thread 1 ---
Thread 1 counting up: 0
Thread 1 counting up: 1
Thread 1 counting up: 2
Thread 1 counting up: 3
Thread 1 counting up: 4
Thread 1 counting up: 5
Thread 1 counting up: 6
Thread 1 counting up: 7
Thread 1 counting up: 8
Thread 1 counting up: 9
Thread 1 counting up: 10
Thread 1 counting up: 11
Thread 1 counting up: 12
Thread 1 counting up: 13
Thread 1 counting up: 14
Thread 1 counting up: 15
Thread 1 counting up: 16
Thread 1 counting up: 17
Thread 1 counting up: 18
Thread 1 counting up: 19
Thread 1 counting up: 20

--- Counting down Thread 2 ---
Thread 2 counting down: 20
Thread 2 counting down: 19
Thread 2 counting down: 18
Thread 2 counting down: 17
Thread 2 counting down: 16
Thread 2 counting down: 15
Thread 2 counting down: 14
Thread 2 counting down: 13
Thread 2 counting down: 12
Thread 2 counting down: 11
Thread 2 counting down: 10
Thread 2 counting down: 9
Thread 2 counting down: 8
Thread 2 counting down: 7
Thread 2 counting down: 6
Thread 2 counting down: 5
Thread 2 counting down: 4
Thread 2 counting down: 3
Thread 2 counting down: 2
Thread 2 counting down: 1
Thread 2 counting down: 0

Both threads have completed their counting without errors.

P:\CSU projects\Programming-3-CSC450\Programming-3-cpp\Modul
e-7 Portfolio Milestone\x64\Debug\Module-7 Portfolio Milesto
ne.exe (process 28348) exited with code 0.
To automatically close the console when debugging stops, ena
ble Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .

```

As shown in Figure 3 the program runs without any issues displaying the correct outputs as expected.